Code-Layout, Readability and Reusability

Date	7 July 2026
Team ID	
Project Name	Personal Network Assistant
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of structure, readability, and reusability. The project uses a separated FastAPI backend, Streamlit frontend, service layer, SQLAlchemy persistence, and pytest coverage.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	Python files consistently use standard indentation.
2	Proper File Structure	Files and folders are logically organized	Yes	Backend routes/services/models, frontend pages, tests, Docker, and docs are separated.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Names such as current_user, confidence_scores, networking_goal, and auth_headers are descriptive.
4	Function / Method Names	Functions clearly describe their behavior	Yes	Functions such as generate_topics, extract_event_themes, log_feedback, and get_current_user are clear.
5	Code Comments	Comments explain important logic	Yes	Module docstrings and comments explain auth, rate limits, model failures, and frontend state.
6	Modular Design	Code is separated into reusable units	Yes	Routes orchestrate reusable auth, profile, analyzer, generator, scorer, history, and feedback services.
7	No Redundant Code	Avoids unnecessary repetition	Partial	Shared frontend helpers reduce repetition, but route and UI request handling still repeat some patterns.
8	Error Handling	Critical operations handle failure safely	Yes	Global handlers, validation, auth errors, model-unavailable 503s, and fallback fact-check responses are implemented.

Reusable Components / Modules:

S.No	Module / Component	Technology	Where Reused	Reusability Level
1	app.auth	Python, JWT, bcrypt	Registration, login, token validation dependency	High
2	app.dependencies.get_current_user	FastAPI dependency	All protected API routes	High
3	app.services.profile_service	SQLAlchemy service	Profile GET/PUT and conversation personalization	Medium
4	app.services.event_analyzer / topic_generator	Transformers, Gemini API	Analyze-event and generate-conversation pipeline	High
5	frontend.common	Streamlit, requests	Home, Profile, Fact Check, History, Feedback pages	High

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	4	Clear backend, frontend, service, model, route, test, and deployment separation.
Readability	4	Descriptive names and extensive docstrings make the flow understandable.
Reusability	4	Reusable services and frontend helpers are present; a few request-handling patterns remain repeated.
Documentation / Comments	4	README, setup notes, module docstrings, and test comments are strong.
Overall Score	4	Well-structured and maintainable local application with room for production hardening.